# Java DataBase Connectivity (JDBC)

Module 3

# JDBC

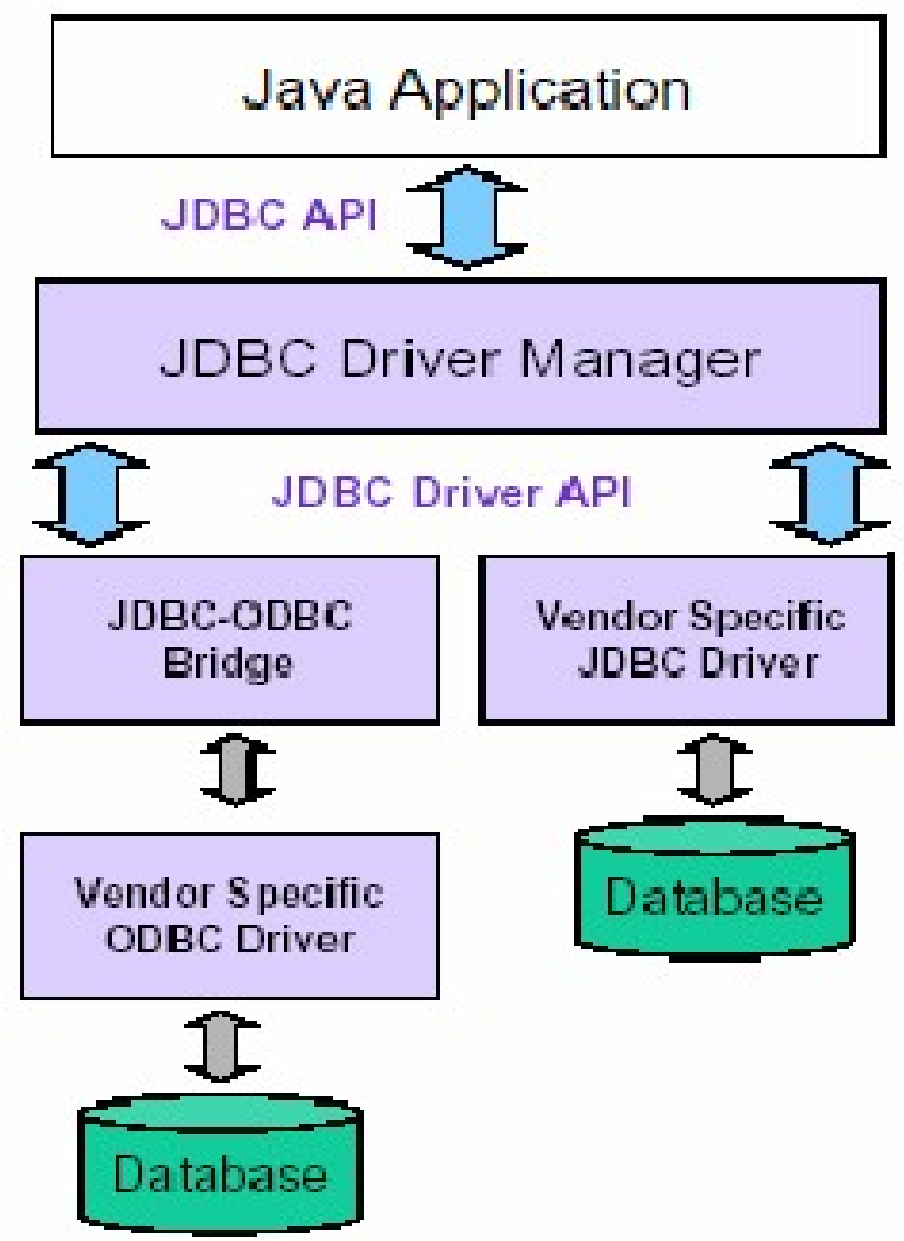
- JDBC is an interface between Java and Database
- JDBC receives queries from Java Application program and communicate with Database
- All the communications are in the form of SQL commands
- JDBC is reponsible for
  - Open a Connection
  - Communicate with database
  - Execute SQL statements
  - Retrive query results

# JDBC

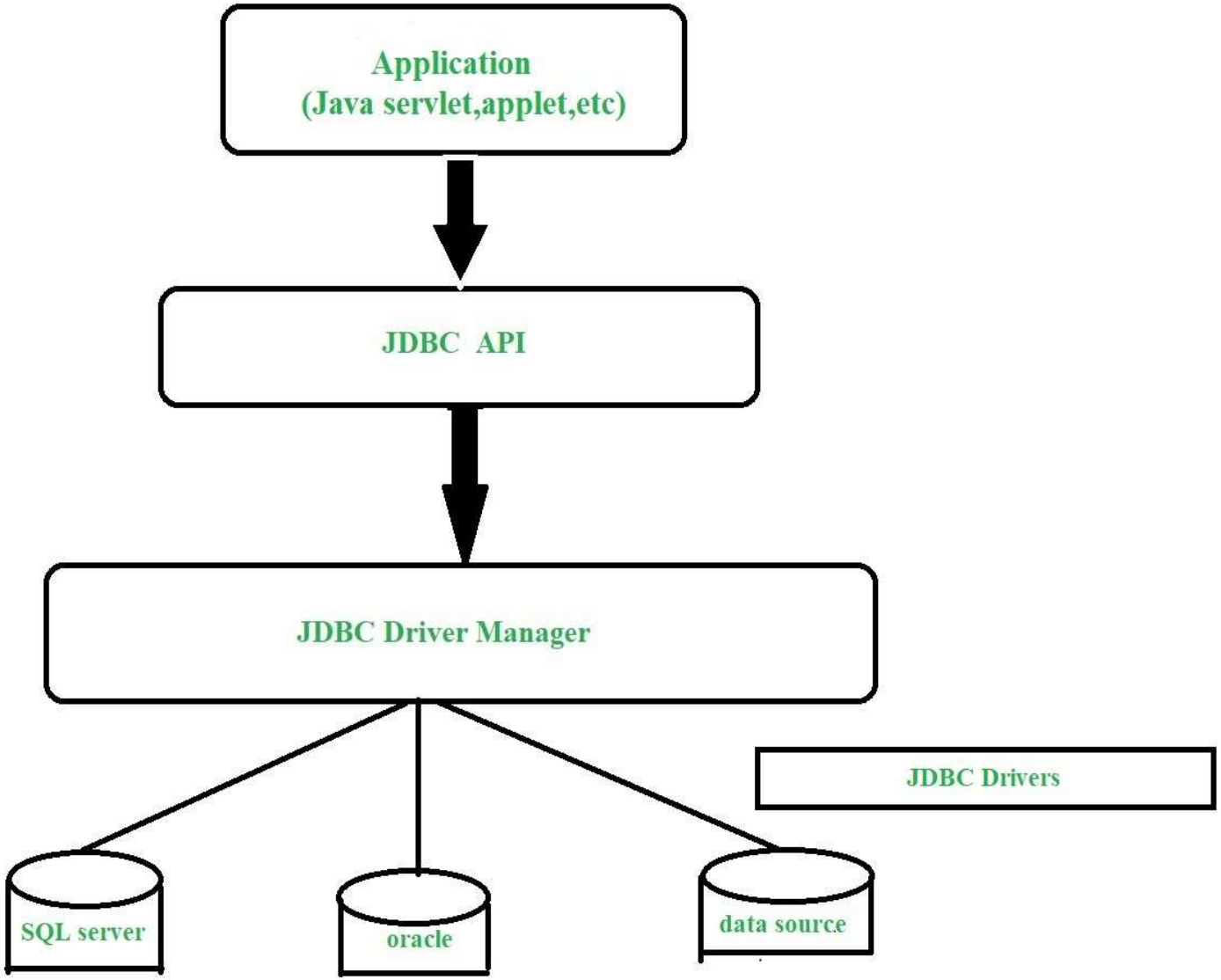
## Features of JDBC

- **Platform Independent:** Write once, run anywhere (Java-based).
- **Database Independent:** Works with any relational database using drivers.
- **Standard API:** Provides a standard set of interfaces (Connection, Statement, ResultSet, etc.).
- **Secure:** Uses Java security model and exception handling.
- **Easy to Use:** Simplifies database connectivity with simple method calls.
- **Supports SQL:** Can execute SQL queries directly from Java code.

# General Architecture



# General Architecture



# General Architecture

JDBC architecture consists of two main layers

1. **JDBC API (Application Layer)**
2. **JDBC Driver API (Driver Layer)**

# General Architecture

- **Application:** It is a java applet or a servlet that communicates with a data source.
- **The JDBC API:**
  - Java Database Connectivity (JDBC) is an application programming interface (API) for the Java programming language which defines how a client may access a database.
  - It is a **Java-based data access technology used for Java database connectivity.**
  - It is part of the Java Standard Edition platform, from Oracle Corporation.
  - It provides methods to query and update data in a database, and is oriented toward relational databases.
  - A JDBC-to-ODBC bridge enables connections to any ODBC-accessible data source in the Java virtual machine (JVM) host environment.

# General Architecture

- **The JDBC API:**
  - Using the JDBC API, you can access virtually any data source, from relational databases to spreadsheets and flat files.
  - JDBC technology also provides a common base on which tools and alternate interfaces can be built.
  - The JDBC API is comprised of two packages:
    - java.sql : Provides the API for accessing and processing data stored in a data source
    - javax.sql: extends the functionality of the JDBC API from a client-side API to a server-side API,
  - The JDBC API allows Java programs to execute SQL statements and retrieve results.
  - Some of the important interfaces defined in JDBC API are as follows: Driver interface , ResultSet Interface , RowSet Interface, Prepared Statement interface, Connection interface, and Classes defined in JDBC API are as follows: DriverManager class, Types class, Blob class, clob class.

# General Architecture

- **JDBC Classes and Interfaces**

| Class/Interfaces | Description |
|---|---|
| DriverManager | Manages JDBC drivers and establishes database connections. |
| Connection | Represents a session with a specific database. |
| Statement | Used to execute static SQL queries. |
| PreparedStatement | Precompiled SQL statement, used for dynamic queries with parameters. |
| CallableStatement | Used to execute stored procedures in the database. |
| ResultSet | Represents the result set of a query, allowing navigation through the rows. |
| SQLException | Handles SQL-related exceptions during database operations. |

# General Architecture

- **Driver Manager:**
  - A class in Java that manages the set of Java Database Connectivity (JDBC) drivers that an application can use.
  - It acts as a layer between the user and the drivers, handling tasks such as:
    - **Establishing a connection between a database and the appropriate driver**
    - **Keeping track of available drivers**
    - **Managing driver login time limits**
    - **Printing log and tracing messages**
  - It plays an important role in the JDBC architecture. It uses some database-specific drivers to effectively connect enterprise applications to databases.
    - The DriverManager class in Java provides the basic service for managing a set of Java Database Connectivity (JDBC) drivers. It maintains a list of database drivers.
    - The DriverManager class is part of the java.sql package. It is a singleton class. Only one instance of this class is created in a JVM.
    - The DriverManager class is used as part of the JDBC API via the static getConnection() method.

# General Architecture

- **JDBC drivers:** To communicate with a data source through JDBC, you need a JDBC driver that intelligently communicates with the respective data source.
- DriverManager manages the set of Java Database Connectivity (JDBC) drivers that are available for an application to use. Applications can use multiple JDBC drivers concurrently if necessary. Each application specifies a JDBC driver by using a Uniform Resource Locator (URL).
- A JDBC (Java Database Connectivity) driver is a software component that allows Java applications to interact with databases. It acts as a bridge between the Java application and the database, translating Java calls into database-specific commands.

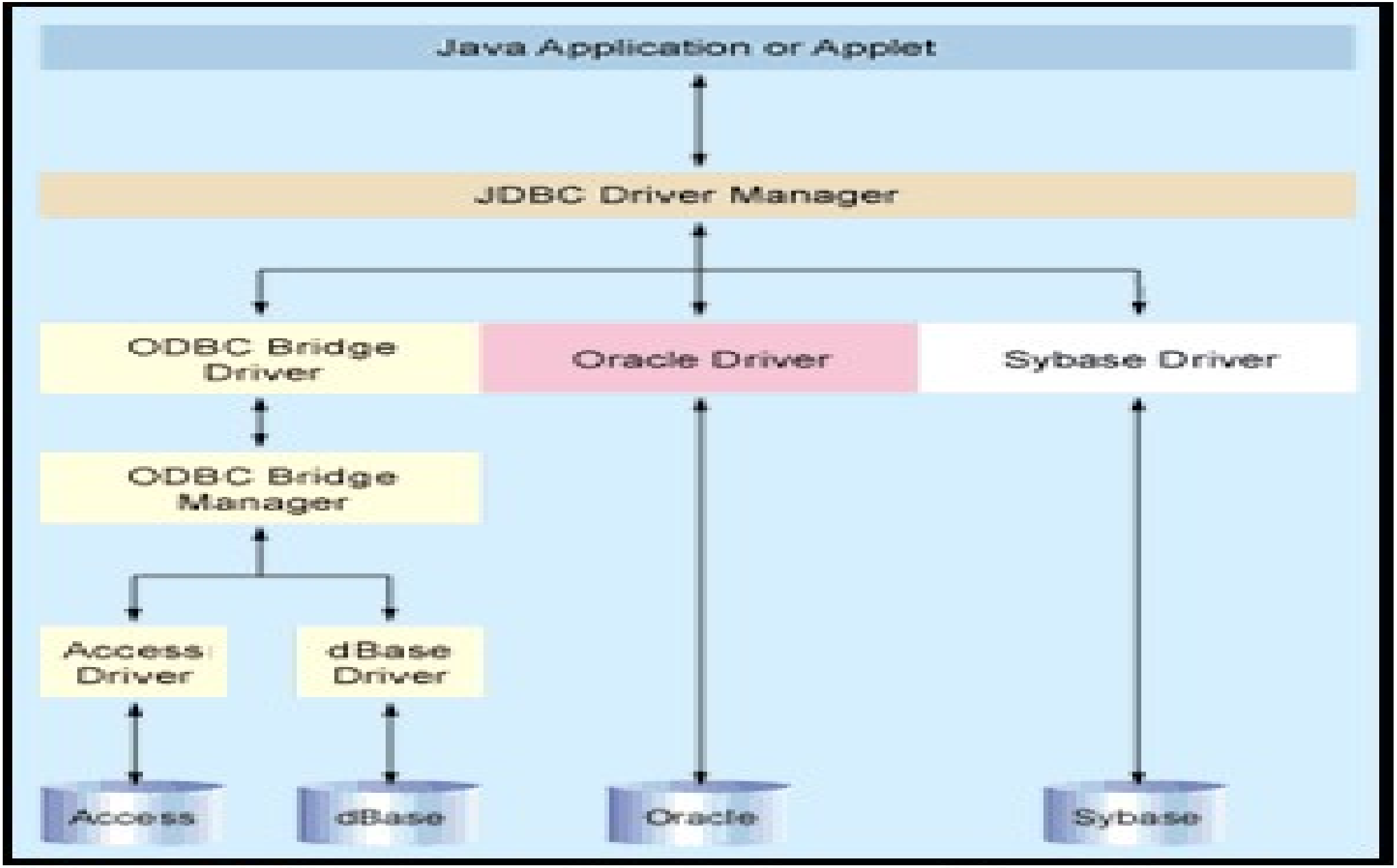



**Figure 1. Anatomy of Data Access.** The Driver Manager provides a consistent layer between your Java app and back-end database. JDBC works natively (such as with the Oracle driver in this example) or with any ODBC datasource.

# Open Data Base Connectivity (ODBC)

- Standard designed by Microsoft to interact with databases
- ODBC is packed with many features and extending support for all type of databases
- ODBC provides multiple mechanism for performing single task and number of data handling capabilities

# JDBC vs ODBC

- JDBC and ODBC are X/OPEN call level interface for SQL
- JDBC is not a derivative of ODBC
- JDBC is compact and simple
- JDBC is meant for simple access to the database and difficult task at least make possible

# JDBC Drivers

- JDBC driver is responsible for making connection with different databases
- It is also translating the queries received from Application and submit into database
- A reverse translation is also required to perform by the Driver
- JDBC Driver speaks JAVA to Application and native language to database
- JDBC Drivers are exists for almost all databases
- Appropriate driver will load for requied database

# JDBC and ODBC

- JDBC (Java Database Connectivity) and ODBC (Open Database Connectivity).
- These both are API standards used for connecting applications to databases.
- ODBC can be used by various programming languages as it is a general API standard.
- But JDBC is Java-specific and it provides a Java-based interface for database access.
- ODBC drivers are not Java-centric which means it is designed to work with different languages through a common interface.
- JDBC drivers are Java-centric.

# JDBC and ODBC

## Difference Between JDBC and ODBC

| Parameters | JDBC | ODBC |
|---|---|---|
| Full Form | Stands for Java Database Connectivity | Stands for Java Database Connectivity |
| Created By | Introduced by Sun MicroSystems in 1997 | Introduced by Microsoft in 1992 |
| Supports | Uses only Java Programming language | Use any programming language like C, C++ etc. |
| API | Java based API | C-based API |
| Platform | Platform Independent | Platform Dependent |
| Lanuage | JDBC is Procedural | ODBC is Object Oriented |
| Applications | Specially designed for Java Applications | It is designed for any applications written in C, C++, Java, Perl etc. |
| Requires | It requires Java Virtual Machine (JVM) | It does not requires. |
| Comparison | Slower in performance as compared to ODBC | Faster in performance as compared to ODBC |

# JDBC

- The Java Development Kit (JDK), which contains the JDBC API **interfaces**, as well as the JDBC **driver manager**
- The JDBC **driver** for a particular brand of relational database
- The JDBC API **interfaces** allow Java applications to access data in a uniform, vendor-independent fashion.
- The JDBC **driver manager** is responsible for finding and loading the correct JDBC driver based on the connection string.
- The JDBC **driver** contains classes that implement the standard JDBC interfaces. The JDBC driver is responsible for actually communicating with the database and carrying out SQL commands and queries.

# JDBC

- JDBC drivers can be implemented in four ways:

  1. **JDBC-ODBC bridge plus ODBC driver**. The JDK contains a JDBC-ODBC bridge, which takes JDBC calls from a Java application and turns them into ODBC calls, utilizing the ODBC driver that is installed on the same machine.

  2. **Native-API partly-Java driver.** This kind of driver takes JDBC calls from a Java application and turns them into calls to the database vendor's native client access software that is installed on the same machine.

# JDBC

- JDBC drivers can be implemented in four ways:

  **3. JDBC-Net pure Java driver**. This kind of driver takes JDBC calls from a Java application and turns them into a vendor independent net protocol. At the other end of the network, the protocol is translated by a middleware server into a vendor specific protocol.

  **4. Native-protocol pure Java driver**. This kind of driver takes calls from Java applications and turns them into vendor specific protocols.

# JDBC Drivers

- JDBC Driver is a software component that enables java application to interact with the database. There are 4 types of JDBC drivers:

1. JDBC-ODBC bridge driver
2. Native-API driver (partially java driver)
3. Network Protocol driver (fully java driver)
4. Thin driver (fully java driver)

# 1) JDBC-ODBC bridge driver

- The JDBC-ODBC bridge driver uses ODBC driver to connect to the database. The JDBC-ODBC bridge driver converts JDBC method calls into the ODBC function calls. This is now discouraged because of thin driver.

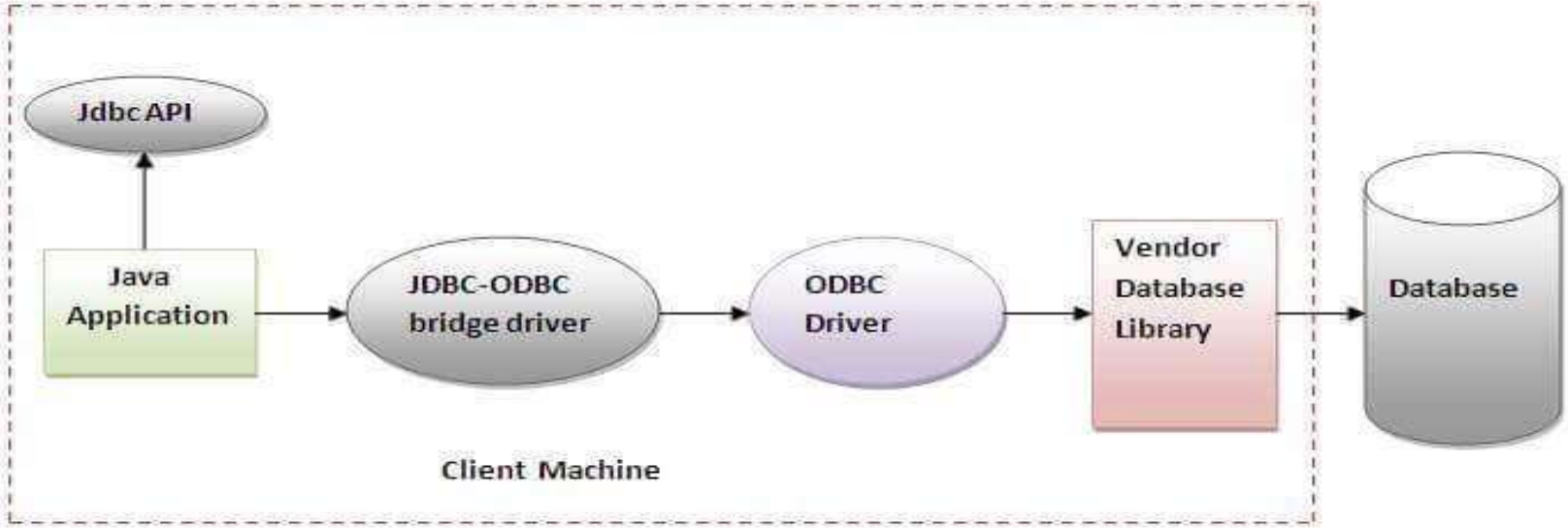



Figure- JDBC-ODBC Bridge Driver

# 1) JDBC-ODBC bridge driver

- Oracle does not support the JDBC-ODBC Bridge from Java 8. Oracle recommends that you use JDBC drivers provided by the vendor of your database instead of the JDBC-ODBC Bridge.

**Advantages:**

- easy to use.
- can be easily connected to any database.

**Disadvantages:**

- Performance degraded because JDBC method call is converted into the ODBC function calls.
- The ODBC driver needs to be installed on the client machine.

# 2) Native-API driver

- The Native API driver uses the client-side libraries of the database. The driver converts JDBC method calls into native calls of the database API. It is not written entirely in java.

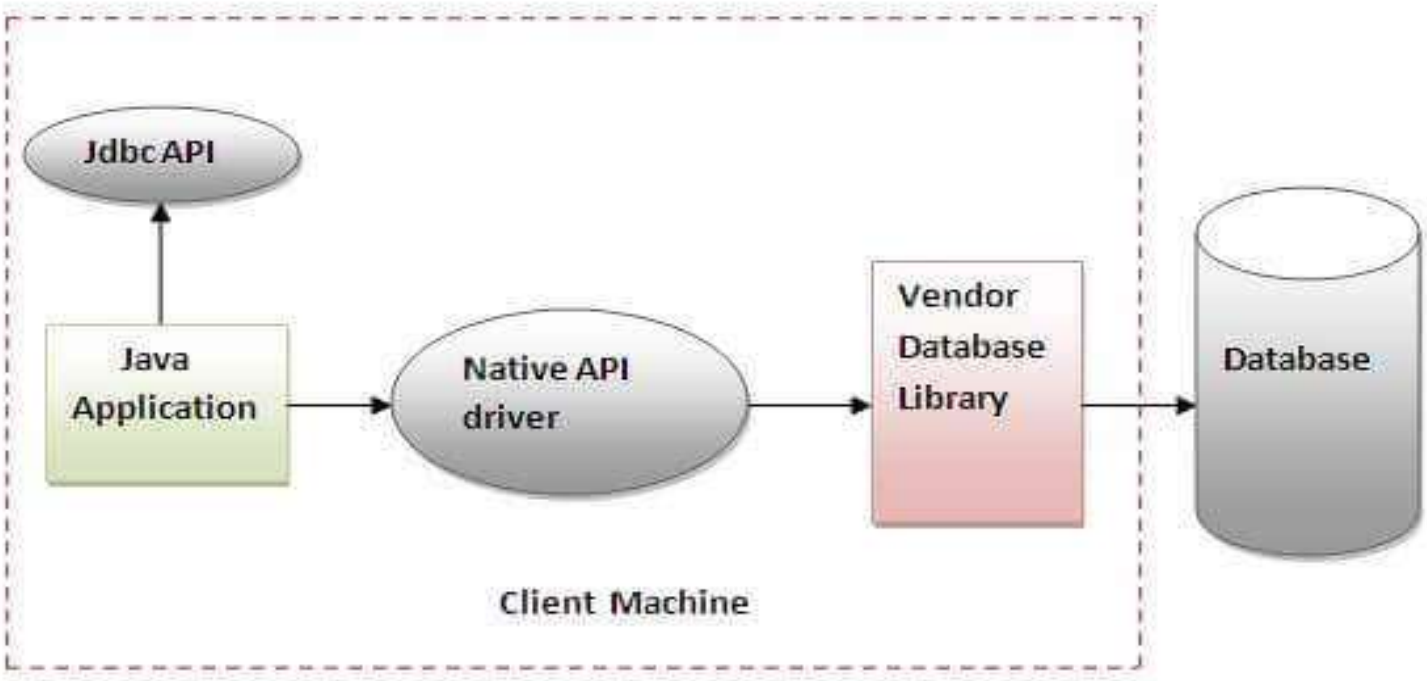



Figure- Native API Driver

# 2) Native-API driver

**Advantage:**

- performance upgraded than JDBC-ODBC bridge driver.

**Disadvantage:**

- The Native driver needs to be installed on each client machine.
- The Vendor client library needs to be installed on client machine.

# 3) Network Protocol driver

- The Network Protocol driver uses middleware (application server) that converts JDBC calls directly or indirectly into the vendor-specific database protocol. It is fully written in java.

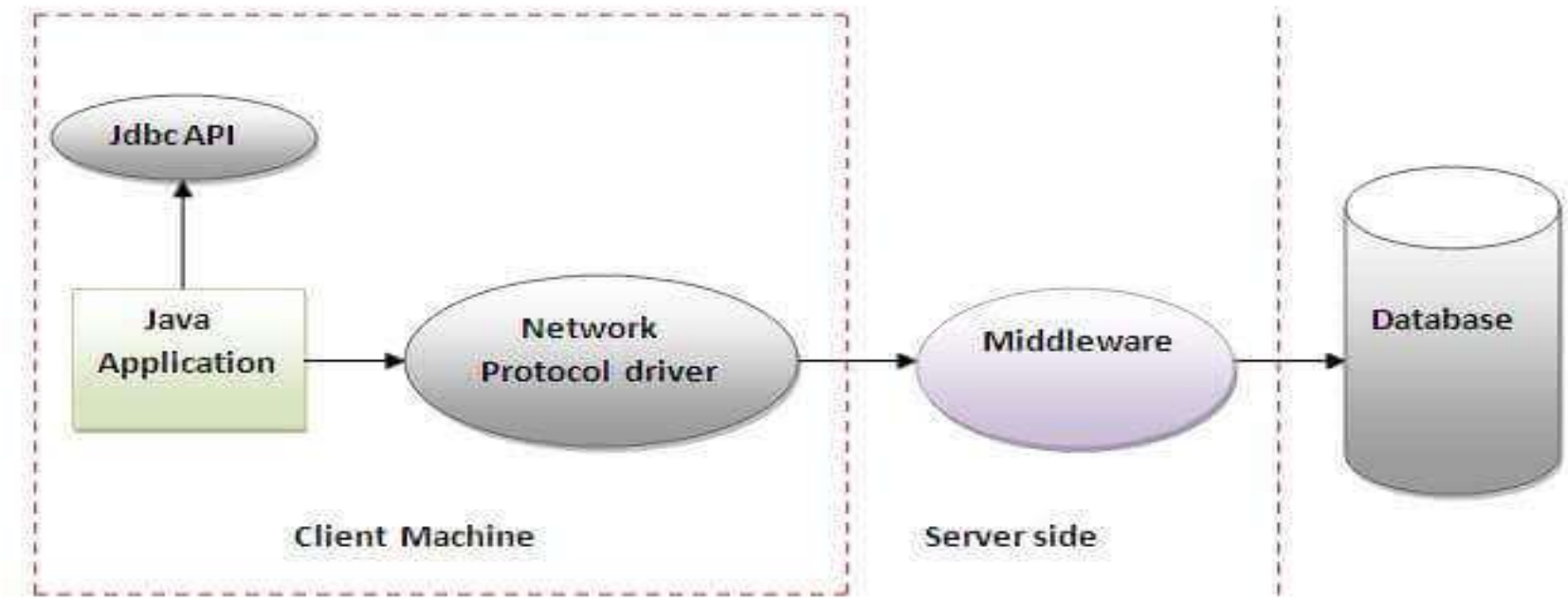



Figure- Network Protocol Driver

# 3) Network Protocol driver

**Advantage:**

- No client-side library is required because of application server that can perform many tasks like auditing, load balancing, logging etc.

**Disadvantages:**

- Network support is required on client machine.
- Requires database-specific coding to be done in the middle tier.
- Maintenance of Network Protocol driver becomes costly because it requires database-specific coding to be done in the middle tier.

# 4) Thin driver

- The thin driver converts JDBC calls directly into the vendor-specific database protocol. That is why it is known as thin driver. It is fully written in Java language.

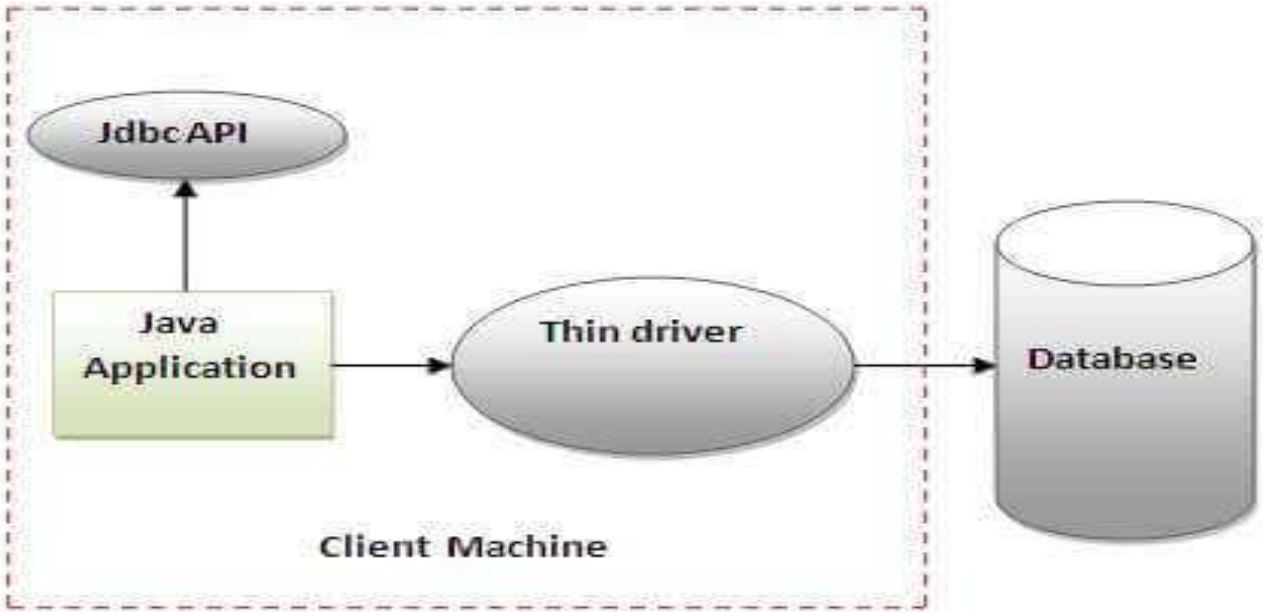



Figure- Thin Driver

# 4) Thin driver

**Advantage:**

- Better performance than all other drivers.
- No software is required at client side or server side.

**Disadvantage:**

- Drivers depend on the Database.

# JDBC Components

The JDBC core comes with the following interfaces and classes:

- ***Driver:*** This is the interface that controls communication with the database server. It also withdraws information associated with driver objects.
- ***Driver Manager:*** It manages any required set of JDBC drivers
- ***Connection:*** This is an interface or session that houses all the methods to connect to any database.
- ***Statements:*** This is used to carry out a static SQL statement
- ***ResultSet:*** This is used to access the result row-by-row

# JDBC Implementation

- Seven Steps
  - Import java.sql package
  - Load and register the driver
  - Establish a connection to the database server
  - Create a statement
  - Execute the statement
  - Retrive the result
  - Close the statement and connection

# Load and Register Driver

Class.forName("Driver ClassName");

Eg:1 Class.forName(“sun.jdbc.odbc.JdbcOdbcDriver”)

2 Class.forName("com.mysql.jdbc.Driver");

Note: Calling the Class.forName automatically creates an instance of a driver and registers it with the DriverManager, so you don't need to create an instance of the class

# Establish Connection

- After registering the driver class, the next step in establishing database connectivity with JDBC is to create a connection. The connection object represents a session between the Java application and the database, allowing it to send SQL statements and retrieve results.
- To create a connection, provide specific details such as the database URL, username, and password.

Connection conn=DriverManager.getConnection("URL");

The drivers loaded recognizes, the JDBC URL in DriverManager.getConnection, that driver establishes a connection to the DBMS specified in the JDBC URL.
The DriverManager class,manages all of the details of establishing the connection

The connection returned by the method DriverManager.getConnection is an open connection you can use to create JDBC statements that pass your SQL statements to the DBMS.

DriverManager.getConnection(“jdbc:mysql://172.16.5.27/campusdb”,”root”,”root”);

```
// Establish a connection to the database
String url = "jdbc:mysql://localhost:3306/mydatabase";
String username = "myuser";
String password = "mypassword";
Connection connection = DriverManager.getConnection(url, username, password);
```

# Managing Statement

- After establishing the database connection, the next step in the process of database connectivity with JDBC is to create a statement object. The statement object enables to execute SQL queries and update statements against the database.
- To create a statement, use the createStatement() method provided by the Connection object.

createStatement() of **Connection** class is used to make object of **Statements**.

Eg: Statement stat=con.createStatement();

Statement object can call **executeQuery**("SQL Command") to execute a select statement.

Use **executeUpdate**("SQL Command") to execute any data updation commands

# Execute Queries

- The next step in establishing database connectivity with JDBC is to execute queries.
- JDBC supports different types of queries, such as SELECT, INSERT, UPDATE, and DELETE.
- Executing queries allows retrieving data from the database or modifying the existing data.
- Now, use the appropriate method provided by the Statement object, such as executeQuery() for SELECT queries or executeUpdate() for other types of queries.

# Execute Queries

```java
// Execute a SELECT query
String query = "SELECT * FROM customers";
ResultSet resultSet = statement.executeQuery(query);

// Process the results
while (resultSet.next()) {
    // Retrieve data from the result set
    String name = resultSet.getString("name");
    int age = resultSet.getInt("age");

    // Do something with the retrieved data
    System.out.println("Name: " + name + ", Age: " + age);
}
```

# Close Connection

- After executing the queries and processing the results, it is important to close the database connection to release the resources and ensure proper cleanup.
- Closing the connection is necessary to free up system resources, close any open database connections, and avoid potential memory leaks.
- To close the connection, call the close() method on the Connection object.

```
// Close the connection and release resources
resultSet.close();
statement.close();
connection.close();
```

# Resultset

- In Java database connectivity, the ResultSet is an interface, and it has a lot of built-in methods for handling the results of SQL queries.
- ResultSet interface represents the result set of a database query by using SQL.
  - When we use PreparedStatement for performing SQL query statements, it will return the PreparedStatement object.
  - Then we call the executeQuery() by using the PreparedStatement object and this result is assigned to the ResultSet object.
  - This ResultSet object contains the result of the SQL query.

# Resultset

- The below statements is representing the syntax of ResultSet with PreparedStatement object.s used for executing the SQL queries.
- 

```
PreparedStatement preparedStatement =
connection.prepareStatement(sql_query);
ResultSet resultSet =
preparedStatement.executeQuery();
```

# Metadata

- Information that describes the structure and properties of your data
- Two types of Metadata
  - Resultset Metadata: Information about the data contained in a Resultset, such as column name, number of columns and column data types
  - Database Metadata: Information about database, such as supported functions, username, current transaction isolation level